Perception System Guide

The perception system includes the following sub modules

- yolov9_ros
- samv2_ros
- sphereformer_ros
- clrernet_ros
- lidar_to_2d

▼ Overview of the modules

YOLOv9 Module Overview

The `yolov9_ros` module provides a real-time object detection and sensor fusion pipeline for autonomous driving applications using ROS (Robot Operating System). It integrates the **YOLOv9** object detection model with LiDAR and radar data to detect, classify, and localize objects in both 2D (camera image space) and 3D (world coordinates).

1. **YOLOv9 Object Detection Node**
    - Subscribes to raw camera images from ROS topics.
    - Runs YOLOv9 inference on GPU (CUDA) or CPU with configurable memory fractions.
    - Filters detections by confidence threshold and suppresses unwanted classes.
    - Handles **traffic light color classification** using HSV color thresholds.
    - Publishes:
        - 2D bounding boxes as `BboxList` messages.
        - Processed images with overlaid detections for visualization.
2. **Objects Transform & Sensor Fusion Node**

- Subscribes to synchronized **LiDAR**, **YOLO bounding boxes**, and **Radar** topics.
- Transforms 2D image bounding boxes into **3D coordinates** using LiDAR point clouds and camera projection matrices.
- Applies **DBSCAN clustering** to filter duplicate camera detections.
- Fuses radar detections with camera detections based on spatial proximity.
- Publishes a combined `BoundingBoxArray` message with:
  - Camera-based 3D object positions.
  - Radar-only detections beyond camera range.

SAMv2 Module Overview

The `samv2_ros` module integrates the **Segment Anything v2 (SAM2)** model into a ROS-based autonomous driving pipeline for **road segmentation** and **3D boundary reconstruction**. It combines real-time image segmentation with LiDAR point cloud data to detect drivable road areas and transform 2D road boundaries into 3D space.

1. Samv2ImageSegmentation Node

- **Input:**
  - Raw camera images (`/camera/raw`)
  - YOLO object bounding boxes for masking obstacles
- **Processing:**
  - Resizes images for efficient inference.
  - Runs **SAM2** model to segment road regions.
  - Cleans segmentation masks with morphological operations.
  - Extracts **left** and **right road boundaries** using contour detection and horizontal binning.
  - Removes points too close to the image boundary.

- **Output:**
  - Segmentation mask images (`/samv2/segmentation_mask`)
  - Left and right road boundaries (`/samv2/left_contour`, `/samv2/right_contour`)

2. Samv2MaskTransform Node

- **Input:**
  - Projected LiDAR point clouds with pixel coordinates.
  - Left/right road contours from `Samv2ImageSegmentation`.
- **Processing:**
  - Matches 2D contour points with corresponding 3D LiDAR points using a **KD-Tree**.
  - Filters points within defined 3D spatial limits.
- **Output:**
  - 3D point clouds for left and right boundaries (`/samv2/left_boundary`, `/samv2/right_boundary`)

Clrernet Module Overview

The `clrernet_ros` module integrates the **CLRerNet** lane detection model with ROS to provide **2D lane detection** and **3D lane reconstruction** for autonomous driving applications. It combines camera images and LiDAR point clouds to deliver accurate left/right lane boundaries and centerlines in real-world coordinates.

1. ClrernetLaneDetection Node

- **Input:**
  - Raw camera images from ROS (`/camera/raw`).
- **Processing:**
  - Runs **CLRerNet** model inference for lane detection.
  - Visualizes detected lanes on the image.

- Publishes:
  - **Lane masks** as images for debugging (`/clrernet/lane_mask`).
  - **All detected lane points** in 2D pixel coordinates (`/clrernet/all_lanes`).

2. Clrernet_Lane_Transform Node

- **Input:**
  - 2D lane points from `ClrernetLaneDetection`.
  - **LiDAR 2D projection** points from sensor fusion.
- **Processing:**
  - Matches 2D lane points with 3D LiDAR points using a **KD-Tree** nearest-neighbor search.
  - Extracts **left** and **right lane boundaries** and computes the **centerline** in 3D space.
  - Applies **temporal smoothing** (EMA) for stable lane trajectories.
- **Output:**
  - 3D left/right lane boundaries (`/clrernet/left_lane`, `/clrernet/right_lane`).
  - 3D centerline (`/clrernet/centerline`).

SphereFormer Module Overview

The `sphereformer_ros` module provides **real-time LiDAR semantic segmentation** and **road boundary extraction** using the **SphereFormer** deep learning model within the ROS (Robot Operating System) framework. It processes 3D LiDAR point clouds to segment the road surface, extract left/right lane boundaries, and publish results for downstream autonomous driving tasks.

1. **Model Integration (SphereFormer)**
   - Loads **SphereFormer**, a 3D spherical transformer network for LiDAR point cloud semantic segmentation.
   - Supports **mixed precision inference** for faster processing on GPUs.

- Reads configurations from YAML and loads pretrained weights for the SemanticKITTI dataset.

2. **ROS Interface**
   - **Subscribers:**
     - Receives filtered LiDAR point clouds from the `ring_filtered_points` topic.
   - **Publishers:**
     - Segmentation masks for all road points.
     - Extracted **left** and **right** lane boundary point clouds.
   - Publishes results in **PointCloud2** format for easy visualization in RViz.
3. **Core Processing Pipeline**
   - **Preprocessing:**
     - Crops incoming point clouds to a region of interest (ROI).
     - Converts ROS PointCloud2 to NumPy for efficient computation.
   - **Model Inference:**
     - Runs the SphereFormer model to classify each point into semantic classes (e.g., road, sidewalk).
     - Extracts only **road surface points** based on predicted class labels.
   - **Boundary Extraction:**
     - Divides the road points along the X-axis into bins.
     - Finds leftmost and rightmost points in each bin to estimate **left and right road boundaries**.
   - **Optional Clustering:**
     - DBSCAN can be used to filter out outlier points for cleaner boundaries.
4. **Published Outputs**
   - **Full Segmentation Mask:** All road points segmented from LiDAR.
   - **Left Boundary Points:** Points representing the left edge of the road.

- **Right Boundary Points:** Points representing the right edge of the road

Lidar2dTransform Module Overview

`transform.py` python script defines a **ROS node** that converts 3D LiDAR point clouds into a **2D projected representation** for downstream perception tasks like sensor fusion or visualization.

Here's the breakdown:

Purpose

- **Input:** Raw 3D LiDAR point cloud (`PointCloud2`) from ROS topics.
- **Processing:**
  1. Crops points to a defined region of interest.
  2. Downsamples the point cloud for efficiency.
  3. Applies a **rigid body transform** followed by a **2D projection**.
- **Output:** Publishes the 2D-projected points with (x, y, z, u, v) fields as a new `PointCloud2` topic.

Key Components

1. **Config-driven topics**
   - Loads topic names from `topics.yaml` so changes in ROS topics don't require code edits.
   - `LIDAR_TOPIC` → input point cloud.
   - `LIDAR_2D_PROJ_TOPIC` → output projected point cloud.

---

1. **Main Class:** `LidarTo2DProjection`
   - **Publisher**: Sends processed 2D-projected point cloud.
   - **Subscriber**: Listens to incoming 3D LiDAR point cloud data.
   - **Callback (`lidar_callback`)**: Processes data whenever new LiDAR data arrives.

---

1. **Processing Steps**
    - `process_pointcloud`:
        - Converts ROS message → NumPy array (`ros_numpy.numpify`).
        - Crops to region defined by `lim_x, lim_y, lim_z`.
        - Downsamples using **voxel grid filtering** for sparsity.
        - Applies:
            - **Inverse rigid transform** (`T1`) → aligns LiDAR points to a common reference frame.
            - **Projection matrix** (`PROJ`) → projects points onto 2D plane.
        - Filters out points outside image bounds (`u, v`).
    - `voxel_downsample`: Reduces the number of points by grouping them into 3D voxels.
    - `publish_projection`:
        - Combines (x, y, z) with (u, v) coordinates into a new `PointCloud2` message.
        - Publishes at a throttled rate to avoid spamming logs.

---

1. **Performance Enhancements**
    - `@timer` **decorator**: Measures function execution time.
    - `@cython.inline`: Potential optimization if compiled with Cython.
    - **GPU support (`torch.device`)**: Available for future acceleration, but not heavily used here.

---

1. **ROS Integration**
    - Node name: `lidar_to_2d_projection`.
    - Automatically starts when the script runs (`rospy.init_node`).
    - Spins to continuously process new point clouds (`rospy.spin`).

▼ Overview of `topics.yaml`

The `topics.yaml` file serves as a **centralized configuration** for all ROS (Robot Operating System) topics used in the perception, planning, and control pipeline of an autonomous system. By organizing topics in one place, it simplifies integration, maintenance, and updates across different modules of the stack.

Structure

1. `topics` **Section**

   This section defines the **raw sensor data** inputs and **processed outputs** from various perception and planning modules. It includes:

   - **Raw sensor topics**: Cameras, LiDARs, radars, IMU, GPS, etc.
   - **Perception outputs**: Bounding boxes (YOLO), road segmentation masks (SAM, Sphereformer), lane detection (ClrerNet).
   - **Planning outputs**: Local paths, global trajectories, obstacle predictions.
   - **Control outputs**: Vehicle control references, performance metrics.

   Example:

   ```
   raw:
     camera: "/camera_fl/image_color"
     lidar_tc: "/lidar_tc/velodyne_points"
     radar_tracks: "/radar_fc/as_tx/radar_tracks"
   ```

2. `categories` **Section**

   Groups topics into **functional categories** for modular access:

   - `perception_input` → Raw sensors feeding perception models.
   - `perception_output` → Outputs from perception models for downstream tasks.
   - `planning` → Trajectory and FSM (Finite State Machine) planning topics.
   - `controls` → Topics for control commands and performance monitoring.

   Example:

```
perception_input:
  - topics.raw.camera
  - topics.raw.lidar_tc
  - topics.raw.radar_tracks
```

Purpose

- **Centralization**: Keeps all topic names in one file to avoid hardcoding in multiple scripts.
- **Modularity**: Allows each module (perception, planning, control) to reference topics consistently.
- **Flexibility**: Changes in topic names require only a single update here.
- **Organization**: Logical grouping makes it easier to manage a growing number of topics.

Use Cases

- **ROS Nodes** can load this YAML file to subscribe/publish without manually defining topic names.
- **Launch files** can reference these topics to ensure consistency across different simulation or real-world configurations.
- **Developers** can quickly understand data flow across the system by checking this single file.

▼ Bash Scripts

1. check_freq.sh

Monitors the **publishing frequency** of all ROS topics defined in `topics.yaml`.

Functionality:

- Uses `yq` to parse topic names from the YAML file.
- Loops through each topic and runs `rostopic hz` to check average publish rates.

- Prints "No data" if a topic isn’t active or publishing messages.

Use Case:

Helps verify if all required ROS topics are live and publishing at expected frequencies before running perception or planning pipelines.

---

2. disable_perception.sh

Stops all perception-related ROS nodes and processes.

Functionality:

- Defines a function `kill_processes()` to find and kill processes by name.
- Stops YOLOv9 object detection, SAM2 segmentation, SphereFormer LiDAR segmentation, UltraFast lane detection, and transform scripts.
- Ensures a clean shutdown of the perception system.

Use Case:

Quickly halts all perception modules when debugging, restarting, or shutting down the system.

---

3. enable_perception.sh

Starts the entire **perception pipeline** with one command.

Functionality:

- Sets up `PYTHONPATH` and ROS environment.
- Launches each perception module (YOLOv9, SAM2, SphereFormer) in separate terminal tabs using their respective conda environments.
- Optionally launches UltraFast lane detection.
- Runs the Lidar 2D projection transform script.

Use Case:

Provides a one-click way to initialize and start all perception nodes for autonomous driving experiments.

---

4. install_ros.sh

Automates **ROS Melodic** installation and sets up required dependencies.

Functionality:

- Adds ROS repository sources and keys.
- Installs `ros-melodic-desktop`, radar message packages, and `jsk_recognition_msgs`.
- Initializes `rosdep` for dependency management.
- Configures ROS environment variables in `.bashrc`.

Use Case:

Speeds up setting up a new machine or development environment with all ROS dependencies in place.

---

5. record_rosbag.sh

Records ROS topics into a **rosbag** file for later playback and analysis.

Functionality:

- Reads `topics.yaml` to determine topics for given categories (e.g., `raw`, `perception_output`).
- Allows multiple categories as input; defaults to `raw` if none specified.
- Prompts user for metadata like location, vehicle ID, road type, etc.
- Saves both the rosbag file and metadata in a timestamped directory.
- Publishes metadata on a dedicated ROS topic `/rosbag_metadata`.
- Uses `rosbag record` with file size limits and automatic splitting.

Use Case:

Essential for collecting and organizing data during real-world tests for debugging, training, or offline analysis.

Perception Docker Guide